

Improved Priority Exchange Server: Critical Evaluation and Transfer

Milestone 3 — Special Emphasis: Real-Time Systems, HSHL

Sumon Boiddo

BEng in Electronic Engineering

Hochschule Hamm-Lippstadt

Matriculation Number: 2220054

Group B, Team 6

Abstract—This document is the Milestone 3 (Critical Evaluation and Transfer) for the seminar topic *Improved Priority Exchange Server* (IPE). It critically evaluates IPE: its concrete strengths and limitations, its hidden or unrealistic assumptions, its run-time and memory scalability, suitable and unsuitable application scenarios, and proposed extensions and open research questions. The primary reference is Buttazzo, *Hard Real-Time Computing Systems* [1], Chapter 6.

I. CONCRETE STRENGTHS

- **Near-optimal responsiveness, close to EDL.** IPE serves aperiodic (emergency) tasks almost as fast as the optimal EDL server; it reaches near-optimal response times, matching EDL in almost all cases (only rarely is EDL slightly better). It also always runs at the highest priority, so a request is served immediately whenever capacity is available [1].
- **Low run-time overhead.** The heavy idle-time calculation is done offline (the E and D arrays), so at run time IPE only replays the stored result. Its run-time cost is low, comparable to the DPE server [1].
- **No wasted CPU time.** Through the priority-exchange mechanism and spare-time reclaiming, unused server time is lent to periodic tasks and reclaimed later, so the processor is used efficiently.
- **Formal schedulability guarantee.** The periodic tasks' hard deadlines are always safe as long as the total periodic utilization satisfies $U_p \leq 1$; the server automatically uses the remaining bandwidth $1 - U_p$. This is an exact EDF condition, not a heuristic [3].

II. CONCRETE LIMITATIONS AND RISKS

- **High memory cost.** Because IPE precomputes everything offline, it must store the E and D arrays. When the periods are non-harmonic, the hyperperiod (lcm) becomes very large, so the tables become very large and the memory use can become prohibitive — the main risk of IPE [1].
- **Requires a known, static task set.** The periodic task set must be known in advance to precompute the arrays. If the task set changes at run time, the precomputed

plan becomes invalid and the guarantees are lost, so IPE cannot be used in dynamic systems.

- **Requires EDF.** IPE needs an EDF scheduler, so it cannot run on fixed-priority-only (RM) platforms.
- **Implementation complexity.** IPE is more complex than simpler servers (such as TBS), because of the priority-exchange bookkeeping and the offline precomputation step.
- **Not fully optimal.** IPE is only nearly optimal; in rare cases the EDL server is slightly better.

III. HIDDEN OR UNREALISTIC ASSUMPTIONS

- **Fixed, known task set.** IPE assumes the periodic task set is fully known and fixed offline. Real systems are often dynamic (tasks added, removed, or changed at run time), so this assumption may not hold.
- **Single processor.** The basic IPE model assumes a single CPU. Modern embedded systems are often multi-core, and IPE's basic formulation does not extend to them.
- **Accurate WCET.** IPE assumes each task's worst-case execution time is known accurately. In reality, WCET is hard to estimate precisely, and a task's actual execution time may vary from run to run.
- **Full preemption.** IPE assumes any task can be paused (preempted) instantly. Real systems often have non-preemptible sections (critical regions, I/O), which the basic model ignores.
- **Implicit deadlines and synchronous release.** IPE assumes deadline equal to period ($D_i = T_i$) and that all periodic tasks start together at $t = 0$. Real tasks may have different deadlines and arbitrary release times.

IV. SCALABILITY ANALYSIS

- **Run-time cost — scales well (low).** At run time IPE only replays the precomputed replenishments and performs light priority-exchange bookkeeping (comparable to DPE). Because the heavy work is done offline, the run-time cost stays low even as the system grows.
- **Memory cost — scales poorly (high).** The E and D tables must cover one hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$.

As the number of tasks grows, or when the periods are non-harmonic, the hyperperiod can grow explosively, so the tables and the memory needed become very large. Memory is IPE's scalability bottleneck.

- **Offline computation cost.** Computing the EDL idle times offline also grows with the hyperperiod, but this is a one-time design-time cost and does not affect run-time performance.
- **Overall.** IPE scales well in time but poorly in memory. It is best for systems with a small number of harmonically-related tasks (small hyperperiod) and does not scale to large or non-harmonic task sets.

V. SUITABLE APPLICATION SCENARIO

- A good example is a **robot (motor) controller**. It runs fixed periodic control loops — for example reading sensors and updating motor commands at regular, harmonically-related rates — which are known and fixed at design time and have hard deadlines that must never be missed.
- Alongside these, it handles occasional soft aperiodic events such as operator commands, status requests, or diagnostic messages, which should be served quickly but do not have hard deadlines.
- IPE fits this scenario well because the periodic task set is fixed and known offline (so the E and D arrays can be precomputed); the control periods are usually harmonic (so the hyperperiod and tables stay small); the controller runs on a single processor with an EDF scheduler; and IPE guarantees the periodic control deadlines ($U_p \leq 1$) while serving the aperiodic commands with near-optimal speed.

VI. UNSUITABLE APPLICATION SCENARIO

- A poor example is an **automotive engine-control unit (ECU)** running an AUTOSAR-based operating system, which uses fixed-priority scheduling.
- IPE fails here because AUTOSAR is fixed-priority (RM-style) and provides no EDF scheduler, so IPE cannot run at all; and because, if the set of tasks changes at run time, the precomputed E and D arrays become invalid and IPE loses its guarantees. On such a platform a fixed-priority server (Polling, Deferrable, or Sporadic Server) is the appropriate choice instead.
- More generally, IPE is unsuitable for any dynamic system (changing task set), any multi-core platform, or any system with non-harmonic periods on a memory-limited device.

VII. PROPOSED EXTENSIONS AND OPEN QUESTIONS

Proposed extensions.

- **Reduced-memory IPE:** compress or bound the E and D tables (for example by approximating the EDL idle times), so IPE can be used even when the periods are non-harmonic and the hyperperiod is large. This addresses IPE's main limitation.

- **Multi-core IPE:** extend the basic single-processor model to multiprocessor / multi-core platforms.
- **Adaptive IPE:** allow the task set to change at run time by recomputing or incrementally updating the E and D arrays online, so IPE can be used in dynamic systems.
- **Robust IPE:** make IPE tolerant to WCET uncertainty, since worst-case execution time is hard to estimate exactly in practice.

Open questions.

- How can the memory cost of the E and D tables be bounded or reduced for realistic non-harmonic task sets?
- Can IPE be extended to multi-core while keeping its near-optimal responsiveness and its schedulability guarantee?
- Can the offline precomputation be made incremental, so the task set can change at run time without recomputing everything?
- Is there a closed-form worst-case response-time bound for aperiodic requests under IPE?

VIII. MILESTONE 3 CHECKLIST MAPPING

TABLE I
M3 CHECKLIST MAPPING

M3 Requirement	Section
Concrete strengths of IPE	Section 1
Concrete limitations and risks of IPE	Section 2
Hidden or unrealistic assumptions in IPE	Section 3
Runtime, memory, scalability implications	Section 4
Suitable application example for IPE	Section 5
Unsuitable application example for IPE	Section 6
Proposed extensions and open questions	Section 7

REFERENCES

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011, ch. 6 (Dynamic Priority Servers).
- [2] M. Spuri and G. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1994.
- [3] M. Spuri and G. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, no. 2, March 1996.
- [4] J. Lehoczky, L. Sha, and J. Strosnider, "Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1987.
- [5] H. Chetto and M. Chetto, "Some Results of the Earliest Deadline Scheduling Algorithm," *IEEE Trans. Software Engineering*, vol. 15, no. 10, 1989.
- [6] S. Boiddo, *AI Usage Protocol, Milestone 3: Improved Priority Exchange Server*, HSHL, 2026. Protocol version 0.2; JSON and BibTeX files submitted with this milestone.